EXPERIMENT NO.15

Implement Collections Framework using ArrayList, HashMap, HashSet and Iterator

PART B – TO BE COMPLETED BY STUDENTS

Roll No: C-60

Name: Avdhut Parish Madhavi

Class / Division: SE-C

Batch: C2

Date of Experiment: _____

Date of Submission: _____

Grade: _____

B.1 Document Created by the Student

Extend the program to maintain a second HashMap for another healthcare-related attribute and record the observed output:

A second HashMap named bloodGroups is added to store each patient ID along with the patient's blood group. The first HashMap stores patient IDs and names, while the second HashMap stores patient IDs and blood groups. Both maps are displayed using their patient IDs as keys. This demonstrates how HashMap can maintain different attributes for the same patient.

B.2 Student Program / Output

Paste/write the final program and attach or reproduce the observed output below:

Program: CollectionsDemo.java

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;

public class CollectionsDemo {
    public static void main(String[] args) {
        ArrayList<String> patients = new ArrayList<>();
        patients.add("Aarav");
        patients.add("Meera");
        patients.add("Kabir");
```

```
        patients.add("Meera");

        System.out.println("ArrayList:");
        Iterator<String> it = patients.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }

        HashMap<Integer, String> records = new HashMap<>();
        records.put(101, "Aarav");
        records.put(102, "Meera");
        records.put(103, "Kabir");

        System.out.println("HashMap - Patient Names:");
        for (Integer id : records.keySet()) {
            System.out.println(id + " -> " + records.get(id));
        }

        // Second HashMap stores each patient's blood group
        HashMap<Integer, String> bloodGroups = new HashMap<>();
        bloodGroups.put(101, "O+");
        bloodGroups.put(102, "A+");
        bloodGroups.put(103, "B+");

        System.out.println("HashMap - Blood Groups:");
        for (Integer id : bloodGroups.keySet()) {
            System.out.println(id + " -> " + bloodGroups.get(id));
        }

        HashSet<String> uniquePatients = new HashSet<>(patients);
        System.out.println("HashSet:");
        for (String name : uniquePatients) {
            System.out.println(name);
        }

        patients.remove("Kabir");
        System.out.println("After removal: " + patients);
    }
}
```

Output:

```
C:\Java\JAVA PRACTICALS>java CollectionsDemo.java
ArrayList:
Aarav
Meera
Kabir
Meera
HashMap - Patient Names:
101 -> Aarav
102 -> Meera
103 -> Kabir
HashMap - Blood Groups:
101 -> O+
102 -> A+
103 -> B+
HashSet:
Meera
Aarav
Kabir
After removal: [Aarav, Meera, Meera]
```

B.3 Observations and Learning

• Why can ArrayList grow dynamically?

ArrayList automatically increases its capacity when more elements are added, so its size does not need to be fixed in advance like an array.

• When is HashMap more suitable than ArrayList?

HashMap is more suitable when data needs to be stored as key-value pairs and a value must be found using a unique key, such as patient ID.

• Why does HashSet remove duplicates?

HashSet stores only unique elements. If the same value is added more than once, it keeps only one copy of that value.

• What is the role of Iterator?

Iterator provides a standard way to access collection elements one by one using methods such as hasNext() and next().

Student observation / learning:

In this program, ArrayList stores patient names and allows duplicate names. HashMap stores patient details using patient IDs as keys. HashSet keeps only unique patient names, and Iterator traverses the ArrayList elements.

B.4 Conclusion

Patient information management and health data management systems are stated syllabus contexts.

Student conclusion:

The Collections Framework was implemented using ArrayList, HashMap, HashSet and Iterator. A second HashMap was added to store patient blood groups. The program demonstrates dynamic storage, key-value mapping, removal of duplicate values, and traversal of collection elements.

B.5 Questions of Curiosity

1. Why can ArrayList grow dynamically?

ArrayList can grow dynamically because it automatically expands its internal storage when additional elements are added.

2. When is HashMap preferable to ArrayList?

HashMap is preferable when information is stored as key-value pairs and values need to be retrieved using keys, such as patient ID to patient name.

3. Why does HashSet remove duplicate values?

HashSet is designed to store unique elements only, so duplicate values are not stored more than once.

4. What is the role of Iterator?

Iterator is used to traverse a collection one element at a time, commonly using hasNext() to check for another element and next() to retrieve it.